891951, 895227, 897270, 913371

Modular Monolith and Microservices Project Documentation for Amateur Volleyball Tournament

Introduction

The projects aim to develop a web application for managing amateur volleyball tournaments. The system allows the creation of tournaments, registration of teams and players, a managing of match outcomes and the ability for users to book tickets for specific matches. Our aim was to create the web app in a monolithic way, through a modular monolithic architecture, to study its behaviour, test it and go through a quicker development, then move to a microservices architecture to allow properties such as elasticity, performance and scalability. To achieve this we have created two repositories, one where we developed the monolithic version and one in which we developed the distributed one.

Requirements And How To Run

The projects rely on Docker for containerization, allowing standardised execution consistent for each system. Detailed instructions on how to run the projects are present in the README.md files present in the github repository for each version (monolithic and distributed) of the project, but can be summarised for both to:

- Having Docker open.

- Running this command from inside the cloned repository, where the `compose.yaml` is located:

```
docker compose up -d --build
```

  In the microservices version, Docker Compose builds and runs multiple containers, each corresponding to an individual service, using a dedicated Dockerfile per service.

- Followed by opening the frontend application at:
  - For the modular monolith: `http://localhost:8080/`
  - For the microservices: `http://localhost:3000`

Technologies

Backend: Java 21 with Spring Boot (v3.4 for the microservices, v4.0 for monolith). We relied on Java for its robust OOP features and our familiarity with the ecosystem, which was also close to the course material.
Frontend:

- Monolith: Server side rendering using Thymeleaf and HTML.

- Microservices: Client side rendering using React (built with Vite). We couldn't use Thymeleaf here to decouple the client from the distributed backend services.

Database: PostgreSQL
Use of Generative AI: tools like Gemini and Claude were utilized to accelerate the generation of the frontend pages especially, which were then edited, fixed and adapted in order to satisfy the project's requirements. Same techniques were applied for the conversion of the pages to a .jsx format, in order to adapt them for the distributed architecture. Generative AI has also been used to evaluate considerations on design and architectural choices, to provide overviews on possible implementations of different structures and to generate repetitive sections of the code, like simple non-exhaustive examples of unit tests.

Architectures

Monolithic

We chose to initially develop the system using a modular monolith architecture because of simplicity, maintainability, development speed and clear domain boundaries.
The modular monolith, used in the project, is a single deployable application, but internally is well-divided into modules each one for a specific domain (tournaments, teams, users, ticketing).
These modules communicate with each other using functionalities defined into interfaces.

- Modules:

Although only one Postgres database is used, it's divided into different schemas and we limit each module to interact only with its own schema.
The Postgres database schemas are defined in an appropriate file called init.sql which is run at the very beginning.

```SQL
CREATE SCHEMA IF NOT EXISTS teams_module;
CREATE SCHEMA IF NOT EXISTS tournament_module;
CREATE SCHEMA IF NOT EXISTS ticketing_module;
CREATE SCHEMA IF NOT EXISTS users_module;
```

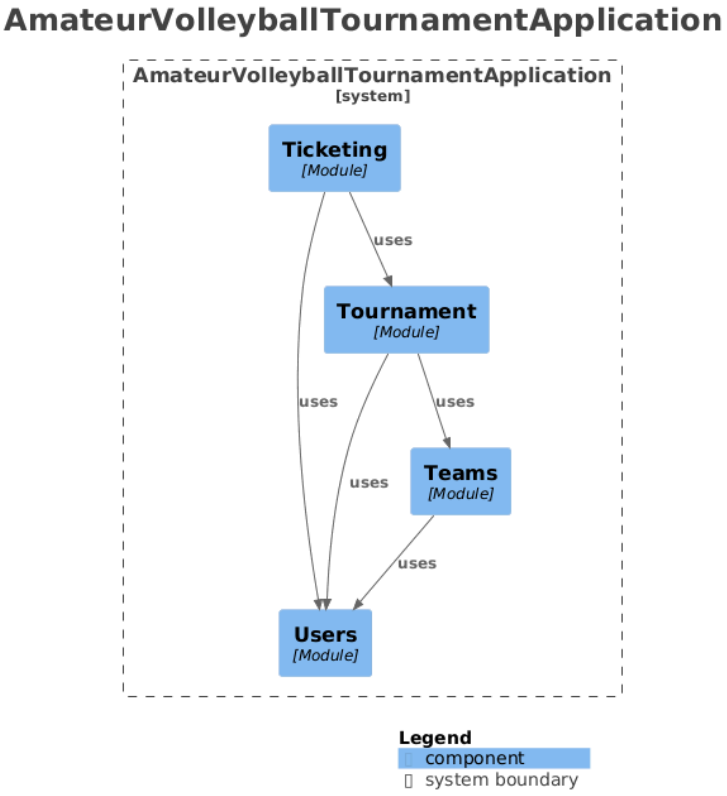
The rest of the application runs in a different and unique container.
Each module has in internal structure of the following packages:

- Web: Contains controllers (like the API endpoints and web controllers)
- Internal: Contains the private implementations (entities, repositories) that are hidden from other modules
- Public Module Interface: The public interface used by other modules and the DTOs exposed to the rest of the application

There is also a test section divided in packages, one for each module, where it is possible to run some tests and perform easy debugging, including the spring-modulith tests to verify proper modularity, preventing cyclic dependencies and ensuring that each module only interacts with the exposed public interfaces.

891951, 895227, 897270, 913371

Another section is in the resources folder where the webpages can be found, divided between each module.
There's also a compose.yaml file which defines the necessary containers to run the entire application and the order in which they will be started.



Microservices

- Why microservices?
We chose this architecture to enforce strict domain boundaries in the design, separated in a way similar to the domain separations used in the modular monolith. This allows to decouple distinct services and enable for example a high concurrency service like 'ticketing' to scale independently of more administrative domains like 'teams'.

- System structure:
We chose a Monorepo development process, that then is built in multiple independent microservices. The system is composed in the backend of services, communicating synchronously via REST for queries and asynchronously via RabbitMQ for orchestration of events like the initial seeding process.

```
// Example REST endpoint:
@PostMapping("/match/stadium")
@RequireRole({"ADMIN"})
public ResponseEntity<String> assignMatchStadium(@RequestBody
TournamentRequests.AssignStadiumRequest request) {
   service.assignStadiumToMatch(request.matchId(),
```

891951, 895227, 897270, 913371

```
request.newStadium());
   return ResponseEntity.ok("Stadium and Capacity assigned
successfully.");
}
```

Each service is a self contained spring boot application with its own dedicated postgres database:

- User Service: manages identities of users, credential validation and role assignment.
- Teams Service: handles the resources related to a team, so it's information, players and stadium.
- Tournament Service: generates and handles the tournament brackets, matches and scores, querying the teams service for team data to build the tournaments.
- Ticketing Service: manages the ticket inventory and bookings, querying the tournament service for information on matches and stadiums seat availability.

There are then infrastructure components to handle routing, persistence and messaging:

- API gateway: a spring cloud gateway acting as single entry point for the frontend and as security guard, validating JWTs and injecting users id and roles into headers trusted by the backend microservices.
- RabbitMQ: asynchronous message broker to handle domain events like the 'ticketSoldEvent' or orchestrate complex systems like the procedural seeding process where each service waits for the others to generate necessary data before starting themselves.
- PostgreSQL: used to implement the four isolated database instances (in the same container for demo development and running resource convenience) ensuring strict access from each service to only its own data and thus acting as a persistence layer.

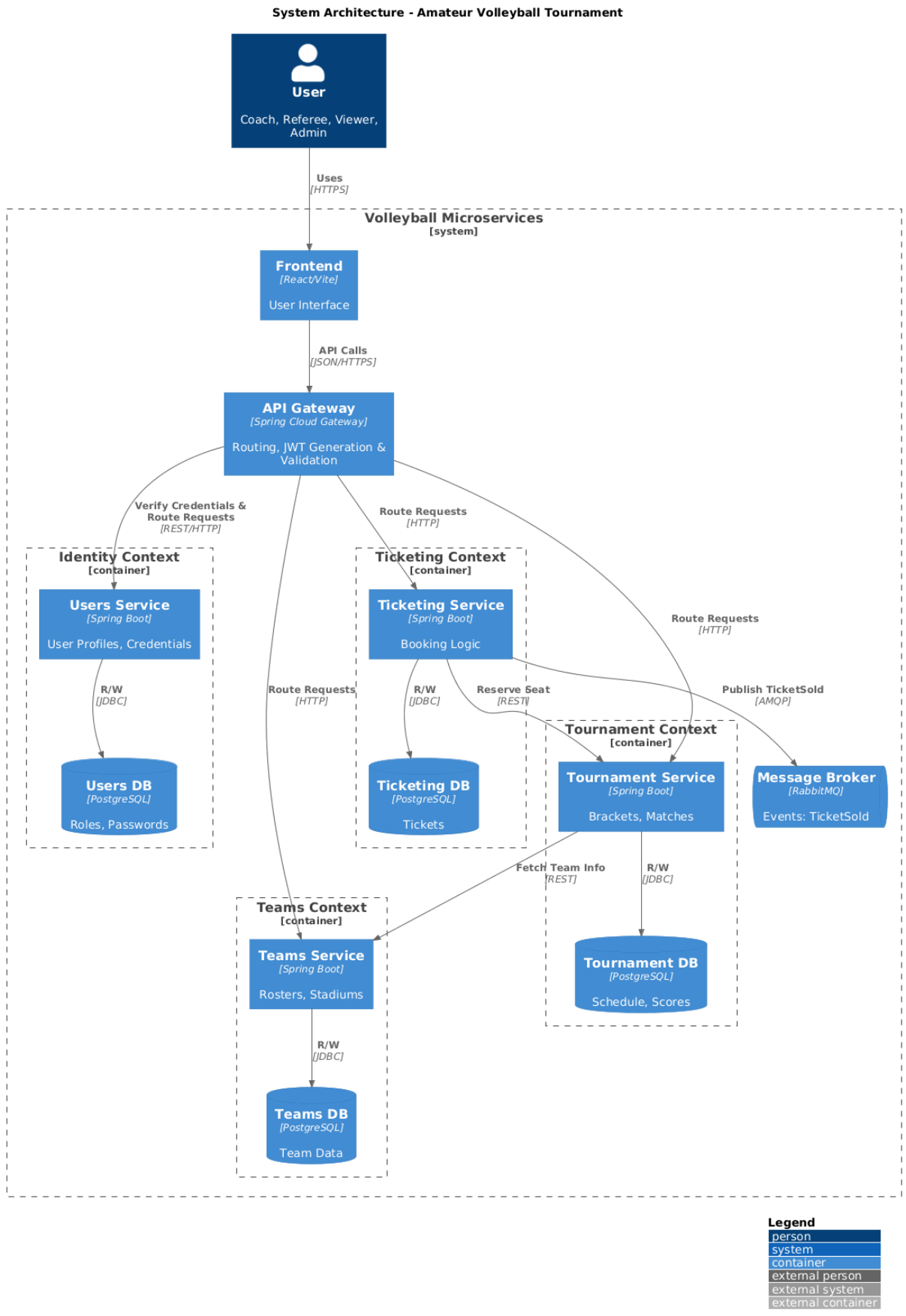
Finally the client-side application:

- A frontend built with React and Vite, interacts exclusively with the API Gateway, unaware of the underlying microservices topology, treating the distributed backend as a single interface.

- Key Choices for the Architecture:
  - Database-per-Service for strict decoupling between each service data.
  - Hybrid Communication: Synchronous (REST) and Asynchronous (Event-Driven)
  - API Gateway and Security:
    - Spring Cloud Gateway as a single entry point.
    - Stateless Authentication as the gateway handles JWT validation and injects the user identity into HTTP headers (trusted by the services, that are thus not exposed themselves to avoid forgeries).
    - Decentralised Authorization as role based access control is enforced locally by within the services using AOP (with the custom @RequireRole annotation), but decoupled by specific frontend logic, as the roles are passed by the api gateway, that could then be adapted to any other kind of user or technical interface.

    - Manual Service Discovery, instead of a heavier complex discovery system, routes locate each other via container aliases.
- Shared Common library module, used to share DTOs, security aspects, enums and rabbitMQ events. This reduces code duplication while also defining the contract of how information (in the shape of the DTOs) is shared between the services.
- Testing Strategy of system/API testing with IntelliJ HTTP client scripts for convenience and an incomplete example of Unit Testing using JUnit and Mockito to test per service in isolation.
- Complete Containerization of the entire ecosystem (services, databases, broker and frontend) via Docker Compose, using spring profiles for dynamic configuration between a development local environment and a completely containerized configuration.

891951, 895227, 897270, 913371



891951, 895227, 897270, 913371

Design patterns

The system strictly adopts the DTO (Data Transfer Object) pattern to manage communication between layers. JPA entities are never exposed directly through REST APIs.

> **Reasons**:
Security: Prevention of accidental exposure of sensitive data (e.g. passwords, hashes, internal metadata).
Stable API contract: Changes to the database schema do not impact API consumers.
Specific validation: Use of separate DTOs for read (UserDTO) and write (InputUserDTO) operations, allowing for different validation rules.

```
// Example DTO:
public record PlayerDTO(String name, int number, VolleyballRole role) {}
```

Technology: Implementation via Java Records to ensure immutability and concise syntax.
Patterns:

- Repository: contains private methods that simulate queries for information gathering inside the DB
- Services: defines methods that can be used by the controllers to manage the logic/business processes. For that, it exposes an interface to hide the implementation.
- Persistence: link between the repository pattern and the service. It is used to call different repositories methods if needed and to format data into DTOs
- Controller: there are of two types: web controller manages access to the static web pages of the application, API controllers expose the endpoints of the requests and call services to retrieve or write data
- Entity: class that describes and simulates the behavior of a DB table
- DTO (Data Transfer Object) pattern to manage communication between layers